Lecture Notes: Neural Network Part II

Course: 2110573 Pattern Recognition

2026-02-11
version 0.1

1 Lecture

Last time we learn about basic Deep Neural Networks (DNNs) and the methods for training and optimizing them. This lecture builds upon the concepts and will look into more specific architectures that are designed for particular types of data and tasks.

2 Neural Network Layers and Architectures

We will explore the following key building blocks and architectures of modern neural networks:

- **Layers:**
  - Convolutional Neural Networks (CNN)
  - Recurrent Neural Networks (RNN), including Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU)
  - Attention mechanisms
- **Architectures:** We will also look at how these layers are combined to create powerful deep learning models.

3 Convolutional Neural Network (CNN)

CNNs are a class of deep neural networks, most commonly applied to analyze visual imagery. They are inspired by the biological processes in the animal visual cortex.

A typical CNN architecture consists of:

- **Convolutional Layers:** These layers apply a set of filters (kernels) to the input to create feature maps. Each filter is designed to detect a specific feature (e.g., edges, corners, textures).
- **Sub-sampling (Pooling) Layers:** These layers reduce the spatial dimensions of the feature maps, which helps to decrease the computational complexity and control overfitting. Common pooling operations include max pooling and average pooling.
- **Fully Connected Layers:** After several convolutional and pooling layers, the high-level features are flattened and fed into fully connected layers for classification or regression.

The output of each convolutional or pooling layer is called a **feature map**.

3.1 Hierarchy of Representations

CNNs learn a hierarchy of representations with increasing levels of abstraction. Each stage acts as a trainable feature transform. For example, in image recognition, the network might learn to detect:

1. **Pixels** (raw input)
2. **Edges**
3. **Textures**
4. **Parts** of objects
5. **Objects**

3.2 Convolution Filters

A convolution filter is a small matrix of weights that slides over the input data (e.g., an image) and performs a dot product. This operation produces a feature map. Each filter learns to detect a specific feature.

3.2.1 Stride

The **stride** is the number of pixels the filter moves over the input matrix at each step. A stride of 1 means the filter moves one pixel at a time.

3.3 CNN Computation Example

Let's walk through a simple convolution operation. Assume we have a 4x4 input image (or feature map) and a 3x3 filter. We will use a stride of 1 and no padding.

Input Matrix (I):

$$I = \begin{pmatrix} 3 & 1 & 0 & 2 \\ 5 & 2 & 1 & 1 \\ 0 & 7 & 3 & 4 \\ 2 & 1 & 1 & 0 \end{pmatrix}$$

Filter/Kernel (K):

$$K = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

The filter slides over the input matrix. At each position, we perform an element-wise multiplication and sum the results to get a single value in the output feature map.

Step 1: Top-Left Position The filter is applied to the top-left 3x3 section of the input.

$$\begin{pmatrix} 3 & 1 & 0 \\ 5 & 2 & 1 \\ 0 & 7 & 3 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = (3*1) + (1*0) + (0*1) + (5*0) + (2*1) + (1*0) + (0*1) + (7*0) + (3*1)$$

$$= 3 + 0 + 0 + 0 + 2 + 0 + 0 + 0 + 3 = 8$$

Step 2: Top-Right Position The filter slides one step to the right.

$$\begin{pmatrix} 1 & 0 & 2 \\ 2 & 1 & 1 \\ 7 & 3 & 4 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = (1*1) + (0*0) + (2*1) + (2*0) + (1*1) + (1*0) + (7*1) + (3*0) + (4*1)$$

$$= 1 + 0 + 2 + 0 + 1 + 0 + 7 + 0 + 4 = 15$$

Continuing this process for all possible positions (bottom-left, bottom-right), we get the final output feature map.

Output Feature Map (O):

$$O = \begin{pmatrix} 8 & 15 \\ 13 & 10 \end{pmatrix}$$

This 2x2 matrix is the new feature map, which can be passed to an activation function and then to the next layer (e.g., a pooling layer).

3.4 Calculating Output and Parameter Sizes

Understanding the dimensions and parameter count of a convolutional layer is crucial for designing CNN architectures.

3.4.1 Output Size Calculation

The spatial dimensions (height and width) of the output feature map can be calculated using the following formula:

$$O = \frac{W - K + 2P}{S} + 1$$

Where:

- O is the output dimension (height or width).
- W is the input dimension.
- K is the kernel (filter) size.
- P is the padding.
- S is the stride.

The depth of the output will be equal to the number of filters used in the layer.

Example: Let's assume an input volume of size **32x32x3** (a small color image). We apply a convolutional layer with the following hyperparameters:

- Number of filters = 10
- Filter size (K) = 5x5
- Stride (S) = 1
- Padding (P) = 2

The output width would be: $O_w = \frac{32-5+2(2)}{1} + 1 = \frac{31}{1} + 1 = 32$. The output height would be the same: $O_h = 32$. The output depth is the number of filters, which is 10. So, the output volume size is **32x32x10**.

3.4.2 Parameter Size Calculation

The number of trainable parameters in a convolutional layer consists of weights and biases.

$$\text{Parameters} = (\text{Weights per filter} + \text{Biases per filter}) \times \text{Number of filters}$$

$$\text{Parameters} = ((K_w \times K_h \times C_{in}) + 1) \times C_{out}$$

Where:

- K_w and K_h are the width and height of the kernel.
- C_{in} is the number of channels in the input volume (its depth).
- C_{out} is the number of filters in the layer (which determines the output depth).
- The '+1' is for the single bias term associated with each filter.

Example (using the same layer as above):

- Kernel dimensions (K_w, K_h) = 5x5
- Input channels $(C_{in}) = 3$
- Output channels $(C_{out}) = 10$

Number of weights per filter $= 5 \times 5 \times 3 = 75$. Total parameters for one filter (weights + bias) $= 75 + 1 = 76$. Total parameters in the layer $= 76 \times 10 =$ **760**.

3.5 Pooling/Subsampling

Pooling layers reduce the dimensions of the feature maps. This makes the model more robust to variations in the position of features in the input image.

- **Max Pooling:** Takes the maximum value from a patch of the feature map.
- **Average Pooling:** Takes the average of the values in a patch.

3.6 Notes on Convolution

- **Shift-Equivariance:** A convolutional layer is shift-equivariant, meaning that if you shift the input, the output will also shift by the same amount.
- **Shift-Invariance:** By using a global max pooling layer after the convolutional layers, the network can achieve shift-invariance. This means that even if the input is shifted, the output remains the same. This is useful for tasks where the position of the object in the image does not matter.

3.7 1x1 Convolution

A 1x1 convolution is a convolution with a filter size of 1x1. It is used for:

- **Dimensionality Reduction:** To reduce the number of feature maps (channels) in the network.
- **Adding Non-linearity:** By applying a non-linear activation function after the 1x1 convolution.

3.8 Advanced Convolutions for Efficiency

Standard convolutions can be computationally expensive, especially as networks get deeper and the number of channels increases. Techniques like using 1x1 convolutions in "bottleneck" layers and depthwise separable convolutions were developed to drastically reduce parameter counts and computational cost.

3.8.1 Bottleneck Layers with 1x1 Convolutions

This technique, popularized by GoogleNet and ResNet, uses 1x1 convolutions to "squeeze" the number of channels down, apply a more expensive convolution, and then "expand" the channels back up.

Let's compare a standard 3x3 convolution with a bottleneck block.

- **Input Volume:** 28x28x256
- **Desired Output:** 28x28x256
- **Convolution:** 3x3 filter

Scenario 1: Standard 3x3 Convolution

- We apply 256 filters of size 3x3x256.
- **Parameters:** $(3 \times 3 \times 256 + 1) \times 256 = (2304 + 1) \times 256 =$ **590,080**.
- **Computational Cost (Multiplications):** Each of the 256 filters performs a convolution over a 28x28 area. The convolution itself has $3 \times 3 \times 256$ multiplications. So, the total is $(28 \times 28 \times 3 \times 3 \times 256) \times 256 \approx$ **461 Million** MACs (Multiply-Accumulate operations).

Scenario 2: Bottleneck Block The block consists of three steps:

1. **Squeeze:** Use a 1x1 convolution to reduce the channels from 256 to 64.
   - Filters: 64 filters of size 1x1x256.
   - Parameters: $(1 \times 1 \times 256 + 1) \times 64 = 16,448$.
   - Output: 28x28x64.
2. **Map:** Apply the 3x3 convolution on the reduced-channel volume.
   - Filters: 64 filters of size 3x3x64.
   - Parameters: $(3 \times 3 \times 64 + 1) \times 64 = 36,928$.
   - Output: 28x28x64.
3. **Expand:** Use a 1x1 convolution to restore the channels to 256.
   - Filters: 256 filters of size 1x1x64.
   - Parameters: $(1 \times 1 \times 64 + 1) \times 256 = 16,640$.
   - Output: 28x28x256.

- **Total Parameters:** $16,448 + 36,928 + 16,640 =$ **70,016**.
- This is a **8.4x reduction** in parameters compared to the standard convolution (70k vs 590k). The computational savings are also significant.

3.8.2 Depthwise Separable Convolution

This technique, used in MobileNet and Xception, factorizes a standard convolution into two much cheaper operations: a depthwise convolution and a pointwise convolution.

Let's use the same example:

- **Input Volume:** 28x28x256
- **Desired Output:** 28x28x256
- **Convolution:** 3x3 filter

Step 1: Depthwise Convolution

- This step applies a *single* 3x3 filter to *each* of the 256 input channels independently. It does not mix channels.
- Filters: 256 filters of size 3x3x1.
- **Parameters:** $(3 \times 3 \times 1) \times 256 = 2,304$. (We can ignore the bias for simplicity here).
- **Output:** 28x28x256. The depth remains the same because each channel was processed independently.

Step 2: Pointwise Convolution

- This is a standard 1x1 convolution used to combine the output from the depthwise step. This is where channel mixing happens.
- Filters: 256 filters of size 1x1x256.
- **Parameters:** $(1 \times 1 \times 256) \times 256 = 65,536$.
- **Output:** 28x28x256.
- **Total Parameters:** $2,304(\text{depthwise}) + 65,536(\text{pointwise}) =$ **67,840**.
- Compared to the standard convolution's **590,080** parameters, this is a **8.7x reduction**. The reduction in computational cost is even more dramatic, making these models ideal for mobile and edge devices.

4 Recurrent Neural Network (RNN)

RNNs are a type of neural network designed to work with sequential data, such as text, time series, and speech. Unlike feedforward neural networks, RNNs have connections that form a directed cycle, allowing them to maintain a "memory" of past information.

4.1 The Need for Memory

In many tasks, such as language modeling, we need to remember past information to understand the present. For example, to understand the sentence "They light the fire," we need to remember the subject "They." A standard DNN framework would process each word independently and would not be able to capture such dependencies.

4.2 RNN Framework

An RNN addresses this by having a recurrent connection. The output of a layer at a given time step is fed back as an additional input to the same layer at the next time step. This allows the network to maintain a state that encodes information about the past.

The new input feature at each time step is a combination of the original input feature and the output of the layer from the previous time step.

4.3 Unrolling and Parameter Sharing

An RNN can be "unrolled" in time, which means we can visualize it as a deep feedforward network where each layer corresponds to a time step. The key property of an RNN is that the same set of weights (parameters) is shared across all time steps. This parameter sharing allows the model to generalize to sequences of different lengths and reduces the total number of parameters to be learned.

4.4 Training RNNs: Backpropagation Through Time (BPTT)

RNNs are trained using a variant of backpropagation called Backpropagation Through Time (BPTT). BPTT unrolls the network in time and then applies standard backpropagation. However, BPTT can suffer from two major problems:

- **Vanishing Gradients:** The gradients can become very small as they are propagated back through many time steps, making it difficult for the network to learn long-range dependencies.
- **Exploding Gradients:** The gradients can become very large, leading to unstable training. This can be mitigated by using gradient clipping.

4.5 Gated Recurrent Unit (GRU)

The GRU is a type of RNN that uses gating mechanisms to control the flow of information. It has two gates:

- **Reset Gate (r):** Determines how much of the past information to forget.
- **Update Gate (z):** Determines how much of the new input to incorporate into the state.

GRUs can help to mitigate the vanishing gradient problem and are simpler than LSTMs.

4.6 Long Short-Term Memory (LSTM)

The LSTM is another type of RNN that is explicitly designed to handle long-range dependencies. It has a more complex structure than a GRU, with three gates and an explicit memory cell:

- **Forget Gate (f):** Decides what information to throw away from the cell state.
- **Input Gate (i):** Decides what new information to store in the cell state.
- **Output Gate (o):** Decides what to output based on the cell state.
- **Cell State (c):** An explicit memory cell that can carry information over long sequences.

4.7 Bi-directional LSTM

In many tasks, it is useful to have information from both the past and the future to make a prediction at the current time step. A bi-directional LSTM consists of two LSTMs: one that processes the sequence from left to right, and another that processes it from right to left. The outputs of the two LSTMs are then concatenated to produce the final output.

5 Attention Mechanism

The attention mechanism is a powerful technique that allows a neural network to focus on specific parts of the input sequence when making a prediction. It was originally developed for machine translation but is now used in a wide range of tasks, forming the basis of the highly successful Transformer architecture.

5.1 Drawbacks of CNNs and RNNs

- **CNNs:**
  - The shift-invariant property can be a disadvantage in tasks where the order of elements is important.
  - They require many layers to capture long-range information.
- **RNNs:**
  - They have poor parallelism because they process the sequence sequentially.
  - They suffer from the vanishing/exploding gradient problem, making it difficult to learn very long sequences. A fixed-size context vector from an RNN has to compress all information from the input, which is a bottleneck.

5.2 The Core Idea: A Dynamic Context Vector

Instead of relying on a single, static context vector that summarizes the entire input, attention allows the model to look back at the entire input sequence at every step. It computes a set of **attention weights** that determine the importance of each input element. These weights are then used to create a dynamic **context vector**, which is a weighted sum of the input elements. This context vector changes at each step, focusing on the most relevant parts of the input for the current task.

5.3 Calculating Attention Scores

The key to attention is calculating the weights. This is done by first computing an **attention score** that measures the relevance of each input (from an encoder) to the current output being generated (by a decoder). There are two main ways to do this:

- **Additive Attention (Bahdanau et al., 2014):** This method uses a small, single-layer feed-forward network to compute the score. It is powerful but can be computationally slower.

  $$\text{score}(h_t, h_s) = v_a^T \tanh(W_a[h_t; h_s])$$

  Here, h_t is the decoder state, h_s is the encoder state, and v_a and W_a are trainable weight matrices.

- **Multiplicative (Dot-Product) Attention (Luong et al., 2015):** This method calculates the score using a simple dot product between the decoder and encoder states. It is faster and more space-efficient, as it does not require an extra feed-forward network.

  $$\text{score}(h_t, h_s) = h_t^T W_a h_s$$

Once scores are computed for all inputs, the **softmax** function is applied to them to get the final attention weights, which are positive and sum to 1.

5.4 The Query, Key, Value (QKV) Model

A highly successful and generalized form of multiplicative attention is the **Query, Key, Value (QKV)** model. This abstraction is at the heart of the Transformer.

- **Query (Q):** Represents the current state or what we are looking for. In a sequence-to-sequence model, this would be the decoder's state at the current time step.
- **Key (K):** Represents the "labels" or identifiers for the input elements. The Query is matched against all Keys to calculate attention scores.
- **Value (V):** Represents the actual content of the input elements. Once attention weights are calculated using the Query and Keys, these weights are applied to the Values to create the context vector.

The most common implementation is **Scaled Dot-Product Attention**, described by the formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here, the dot product between the Query and all Keys (QK^T) computes the scores. They are "scaled" by the square root of the key dimension, $\sqrt{d_k}$, to prevent the gradients from becoming too small during training. The softmax turns scores into weights, which are then used to create a weighted sum of the Values.

5.5 Attention Computation Example (QKV Model)

Let's compute the attention for a single query vector with respect to two key-value pairs, using the Scaled Dot-Product Attention model.

Let the dimension of the keys, d_k, be 2. Let's define our Query, Keys, and Values.

Query (Q): The current state of the decoder.

$$Q = \begin{pmatrix} 1 & 0 \end{pmatrix}$$

Keys (K): Representations of two words from the input sentence.

$$K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \quad \text{(Key 1 and Key 2)}$$

Values (V): The actual embeddings of the two words from the input sentence.

$$V = \begin{pmatrix} 2 & 3 \\ 4 & 5 \end{pmatrix} \quad \text{(Value 1 and Value 2)}$$

Step 1: Compute Scores (QK^T)

$$\text{Scores} = QK^T = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} (1*1+0*1) & (1*0+0*1) \end{pmatrix} = \begin{pmatrix} 1 & 0 \end{pmatrix}$$

Step 2: Scale Scores Divide the scores by $\sqrt{d_k} = \sqrt{2} \approx 1.414$.

$$\text{Scaled Scores} = \begin{pmatrix} 1/1.414 & 0/1.414 \end{pmatrix} = \begin{pmatrix} 0.707 & 0 \end{pmatrix}$$

Step 3: Compute Attention Weights (Softmax)

$$\text{Weights} = \text{softmax}(0.707, 0) = \left(\frac{e^{0.707}}{e^{0.707}+e^0}, \frac{e^0}{e^{0.707}+e^0}\right) \approx \begin{pmatrix} 0.67 & 0.33 \end{pmatrix}$$

Step 4: Compute the Context Vector

$$\text{Context} = \text{Weights} \cdot V = \begin{pmatrix} 0.67 & 0.33 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ 4 & 5 \end{pmatrix}$$

$$\text{Context} = (0.67 \times \begin{pmatrix} 2 & 3 \end{pmatrix}) + (0.33 \times \begin{pmatrix} 4 & 5 \end{pmatrix}) = \begin{pmatrix} 2.66 & 3.66 \end{pmatrix}$$

This context vector is the final output, a blend of the input Values weighted by their relevance to the Query.

5.6 Important Attention Variants

- **Self-Attention:** A crucial innovation where attention is applied to a single sequence. Here, the Query, Key, and Value all come from the same input sequence. This allows each element in the sequence to attend to all other elements, capturing intra-sequence dependencies without using an RNN.
- **Multi-Head Attention:** Instead of computing attention once, this mechanism runs the Scaled Dot-Product Attention multiple times in parallel with different, learned linear projections of the Q, K, and V matrices. Each "head" can learn different aspects of relationships in the data. The outputs are then concatenated and linearly transformed to produce the final result.

6 Neural Network Architectures

We will now look at some of the most influential neural network architectures that have been developed over the years.

6.1 A Brief History of ImageNet Architectures

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) has been a major driver of innovation in computer vision. Here is a brief history of some of the key architectures:

- **LeNet (1998):** One of the earliest CNN architectures, used for handwritten digit recognition.
- **AlexNet (2012):** A deep CNN that won the ILSVRC 2012 by a large margin. It used ReLU activations, dropout, and data augmentation.
- **ZFNet (2013):** An improvement over AlexNet, achieved by tuning the hyperparameters.
- **VGG (2014):** A very deep CNN with a simple and uniform architecture, using only 3x3 convolutional filters.
- **GoogLeNet (Inception v1, 2014):** Introduced the "Inception module," which uses multiple filter sizes in parallel to capture features at different scales. It also used 1x1 convolutions for dimensionality reduction.
- **ResNet (2015):** A very deep network (up to 152 layers) that introduced "residual connections" (or skip connections) to address the vanishing gradient problem. This allowed for the training of much deeper networks.

6.2 Trends After ResNet

After the success of ResNet, research has focused on:

- **More Efficient Convolutional Block Design:**
  - **Bottleneck Block:** Uses 1x1 convolutions to reduce and then restore the number of channels, making the 3x3 convolution more efficient.
  - **Factorized Convolution:** Replaces a large convolution with smaller ones (e.g., a 5x5 convolution with two 3x3 convolutions).
  - **Group Convolution (ResNeXt):** Splits the feature maps into groups and applies a separate convolution to each group.
  - **Depthwise Separable Convolution (Xception, MobileNet):** A very efficient type of convolution that consists of a depthwise convolution (one filter per input channel) followed by a pointwise convolution (a 1x1 convolution).
- **Wider, Deeper, and Denser Networks:**
  - **WideResNet:** Argued that wider and shallower ResNets can be more effective than very deep and thin ones.
  - **ResNetv2:** Improvements to the original ResNet architecture.
  - **DenseNet:** Densely connects each layer to every other layer in a feed-forward fashion, which encourages feature reuse and improves gradient flow.
- **Neural Architecture Search (NAS):**
  - **NASNet:** Used reinforcement learning to search for the best network architecture.
  - **EfficientNet:** Used NAS to find a good baseline architecture and then scaled it up in a principled way to achieve a good balance between accuracy and efficiency.

7 Deep Learning Toolkits

There are many deep learning toolkits available, each with its own trade-offs between customizability and ease of use.

- **Low-Level Libraries:** CUDA, cuDNN, BLAS provide the fundamental building blocks for deep learning on GPUs.
- **Frameworks:**
  - **PyTorch:** Known for its flexibility and ease of use, especially for research.
  - **TensorFlow:** A powerful and scalable framework developed by Google.
  - **Keras:** A high-level API that can run on top of TensorFlow, making it very easy to build and train deep learning models.
  - **MXNet:** An open-source deep learning framework supported by Amazon.
- **High-Level Toolkits:** PyTorch Lightning and DIGITS provide even higher-level abstractions to simplify the training process.

8 Tips for Tuning Neural Networks

Tuning neural networks can be a complex process. Here are some tips that can help:

- **Experience and Intuition:** Often, the best hyperparameters are found through experience and a "feel" for the model's behavior.
- **Start with Published Architectures:** A good starting point is to use architectures and hyperparameters from published papers that have been successful on similar tasks.
- **Search Algorithms:**
  - **Grid Search:** Systematically search through a predefined subset of the hyperparameter space.
  - **Random Search:** Randomly sample from the hyperparameter space. It is often more efficient than grid search.
  - **Heuristic Search:** Use rule-of-thumb to guide the search for hyperparameters.
  - **Genetic Algorithms:** Use evolutionary algorithms to find the optimal set of hyperparameters.
  - **Bayesian Optimization:** A probabilistic model-based approach to find the minimum of a function. Tools like Optuna can be used for this.
- **Model Selection over Neuron Count:** The choice of model architecture (e.g., CNN, RNN) is often more critical than the exact number of neurons in a layer. This is because different architectures have different **inductive biases**, which are assumptions made by the model to learn the target function.
- **Debugging Tips:** For a comprehensive guide on debugging neural networks, refer to Andrej Karpathy's blog post: A Recipe for Training Neural Networks.

8.1 PyTorch Steps

Here are the typical steps for building and training a model in PyTorch:

1. **Set up the Dataloader:** To load and preprocess the data in mini-batches.
2. **Define the Network:** Create a class that inherits from `torch.nn.Module` and define the layers of the network. Initialize the weights.
3. **Set up the Optimization Method:** Choose an optimizer (e.g., Adam, SGD) and a learning rate scheduler.

4. **Training Loop:**
   - **Forward Pass:** Compute the output of the network and the loss.
   - **Backward Pass:** Compute the gradients of the loss with respect to the network parameters and update the weights.